Enterprise Vulnerability Management Dashboard

Project Overview

* Web-based vulnerability management platform for enterprise security teams
* Upload and process vulnerability scan reports (Excel, CSV, JSON)
* Smart worksheet detection for multi-sheet Excel files
* Real-time vulnerability tracking and status management
* Team performance leaderboard with MTTR metrics
* Export capabilities (Excel, PDF)
* Role-based access control (Admin / Viewer)

Tech Stack

Frontend

* React 19 - UI Framework
* TypeScript - Type Safety
* Vite 8 - Build Tool
* TailwindCSS 4 - Styling
* Recharts - Data Visualization
* Lucide React - Icons
* XLSX (SheetJS) - Excel Export
* jsPDF - PDF Generation

Backend

* Python 3.x - Language
* FastAPI - REST API Framework
* Pandas - Data Processing
* openpyxl - Excel Inspection
* httpx - Async HTTP Client
* LangSmith - AI Observability
* JSON File - Database Storage
* Docker + Nginx - Deployment

System Architecture

* BROWSER (React + TypeScript)
* Dashboard | Charts (Recharts) | Upload Modal | Export (XLSX/PDF)
* | REST API (HTTP) |
* BACKEND (FastAPI + Python)
* /api/upload-report | /api/db | /api/leaderboard
* Smart Worksheet Detection (openpyxl) | Data Processing (Pandas)
* Storage: xtelify_db.json

API Endpoints

* GET /api/db - Fetch all vulnerability records
* POST /api/db - Add new vulnerability items
* DELETE /api/db - Delete dataset by UploadBatch
* POST /api/upload-report - Upload & process Excel/CSV/JSON
* POST /api/upload-report-with-sheet - Upload with manual sheet selection
* GET /api/leaderboard - Get team performance rankings
* GET /{path} - Serve frontend static files

Data Flow: Upload Process

* 1. User selects Excel file in browser
* 2. Frontend sends FormData to /api/upload-report
* 3. Backend scans all worksheets using openpyxl (read-only mode)
* 4. Smart detection scores each sheet and selects the best one
* 5. Header row detected by scanning first 15 rows
* 6. Pandas reads selected sheet from header row
* 7. Column mapping applied (ID -> IssueID, CVSSSeverity -> Severity)
* 8. Records normalized and saved to xtelify_db.json
* 9. Frontend reloads to display new data

Smart Worksheet Detection

Positive Scoring	Negative Scoring
* +3 pts: ID, Name, Severity, FindingStatus	* -5 pts: 'Grand Total' in sheet name
* +2 pts: Score, CVSSSeverity, WizURL	* -5 pts: 'Count of' in sheet name
* +1 pt: Other expected columns	* -5 pts: 'Pivot' in sheet name
* +5 pts: More than 100 data rows	* -3 pts: Fewer than 20 rows
* +3 pts: More than 50 data rows	* Auto-selects highest scoring sheet!

Column Auto-Mapping

* Supports multiple column naming conventions from different scanners:

```
ID / IssueID / VulnID / CVE -> IssueID
CVSSSeverity / VendorSeverity / Risk -> Severity
FirstDetected / DetectedDate -> DiscoveredDate
WizURL / Link / Reference -> ReferenceLinks
Remediation / Resolution / Fix -> RecommendedAction
```

Vulnerability Grouping

* Frontend groups vulnerabilities by CVE/ID
* Multiple assets affected by same vulnerability shown together
* Expandable rows to see affected assets
* Highest severity from group displayed
* Status aggregated (Open if any asset is open)

```
Example:
CVE-2024-12345 | Critical | 5 Assets Affected
CVE-2024-67890 | High | 12 Assets Affected
```

Leaderboard & Gamification

Points Calculation

* Critical fix: 100 base pts
* High fix: 50 base pts
* Medium/Low fix: 25 base pts
* Speed Multiplier:
* multiplier = SLA_hours / actual_hours
* Faster fixes = More points!

Tier System

* Elite Guardian: > 1800 pts
* SecOps Specialist: > 1400 pts
* Patch Master: > 1000 pts
* Green Horn: < 1000 pts
* Encourages fast remediation!

Project File Structure

```
xtelify-security-portal-main/
app.py - FastAPI backend
xtelify_db.json - JSON database
package.json - Dependencies
Dockerfile - Docker build
src/
App.tsx - Main React app
main.tsx - Entry point
dist/ - Production build
```

Database Schema

* Each vulnerability record in xtelify_db.json contains:

```
{
"IssueID": "CVE-2024-12345",
"Severity": "Critical",
"Status": "Open",
"AssignedTo": "john.doe@company.com",
"DueDate": "2026-07-22",
"RecommendedAction": "Upgrade to v3.1.5"
}
```

Deployment Options

Development Mode	Production Mode
* Terminal 1 (Backend):	* Option 1: Docker
* uvicorn app:app --reload	* docker build -t xtelify-portal .
* --host 0.0.0.0 --port 8000	* docker run -p 80:80 xtelify-portal
* Terminal 2 (Frontend):	* Option 2: Combined
* npm run dev	* npm run build && uvicorn app:app

Summary

* Unified View: All vulnerabilities in one dashboard
* Smart Processing: Auto-detects data format and columns
* Actionable Insights: Charts, stats, and remediation priorities
* Team Accountability: Leaderboard gamifies security work
* Flexible Export: Excel and PDF reports
* Offline-First: Works without internet (JSON file storage)
* Tech Stack: React + TypeScript + FastAPI + Pandas

Questions?